BiteFixes Enterprise Architecture Manual

Volume 09

API Architecture Specification (AAS)

Document ID: BFA-009

Version: 1.0

Status: Draft

Language: English

Architecture Level: Enterprise

Table of Contents

4. API Gateway

5. REST Standards

6. Endpoint Design

7. Versioning

8. Authentication

9. Authorization

10. Request Standards

11. Response Standards

12. Error Standards

13. Pagination

14. Filtering

15. Sorting

16. Search

Chapter 1

Introduction

The BiteFixes API Platform provides a standardized interface for all external communication.

Every client communicates through the API layer.

Clients include:

- Flutter Mobile App
- Web Portal

- WhatsApp Integration
- Administration Panel
- External ERP Systems
- CRM Systems
- Third-Party Integrations
- Bitey AI Services

Chapter 2

API Philosophy

The API is a contract.

It is **not** an implementation detail.

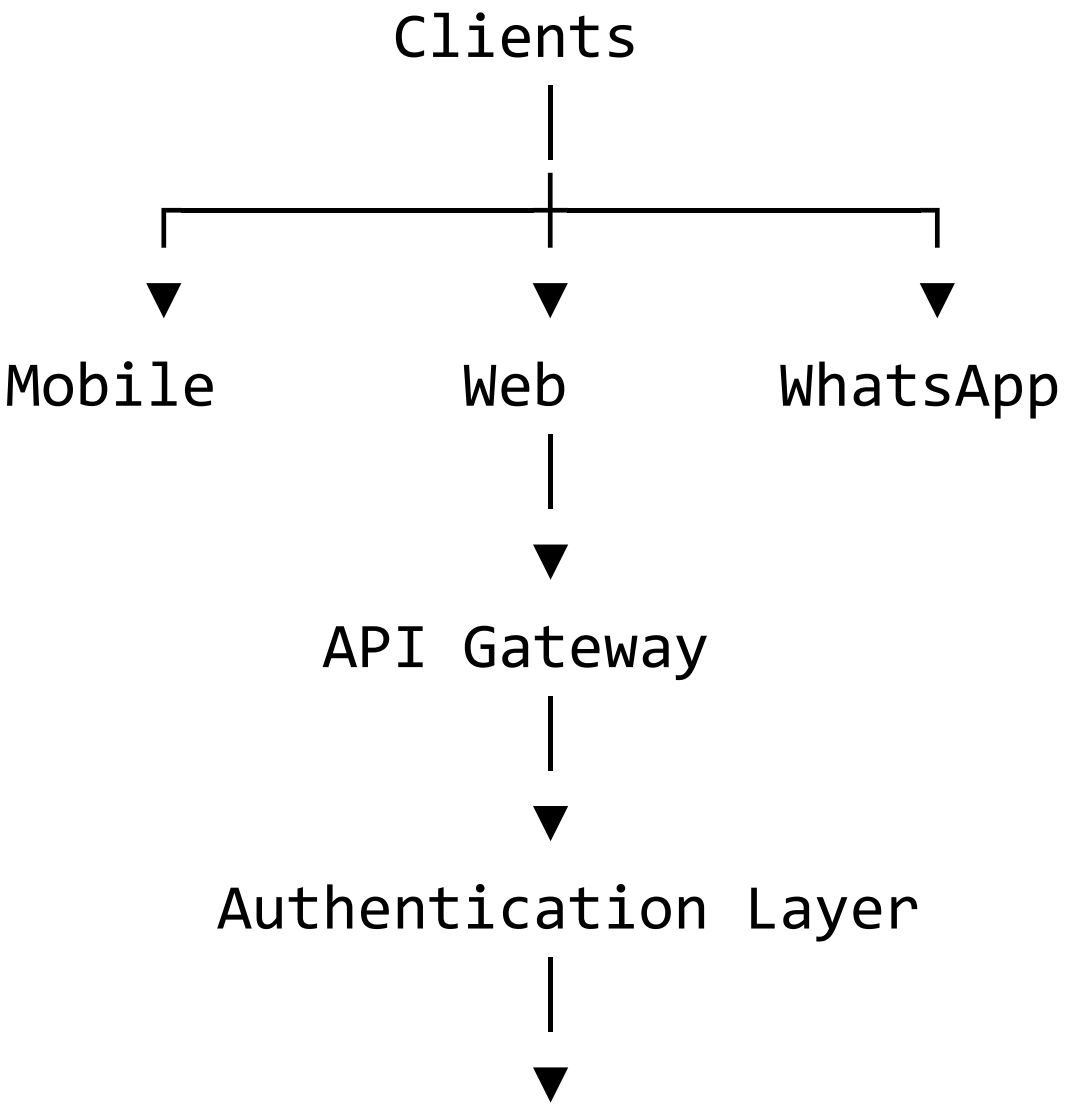
Every endpoint must remain stable and backward compatible whenever possible.

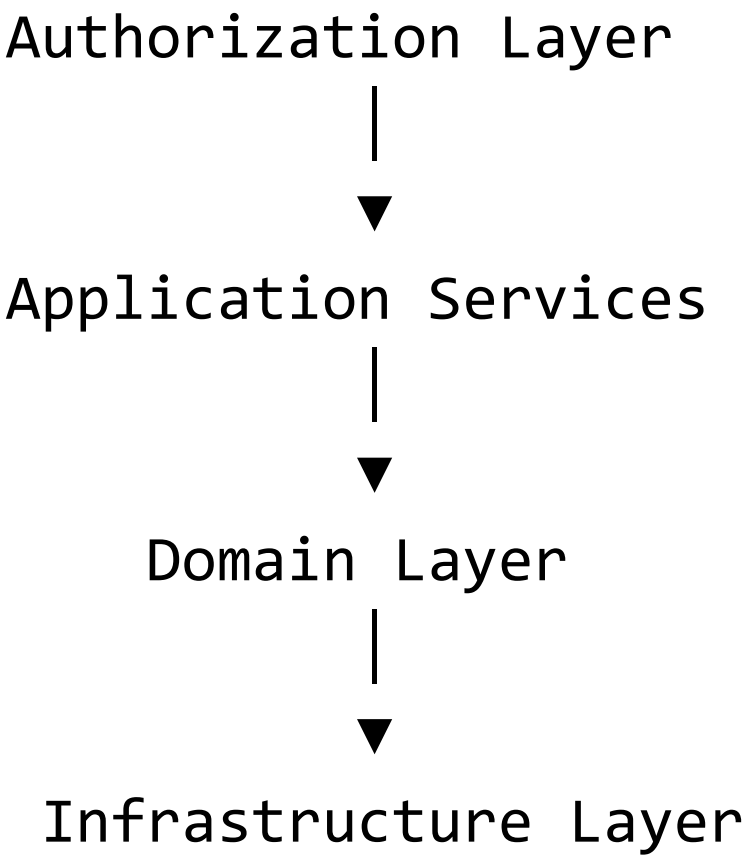
Business rules are never implemented in controllers.

Controllers orchestrate requests and delegate execution to the application layer.

Chapter 3

API Architecture





Chapter 4

API Gateway

The API Gateway is the single entry point to the platform.

Responsibilities include:

- Authentication

- Authorization
- Rate Limiting
- Request Validation
- Logging
- API Version Routing
- Metrics Collection
- Security Headers

No business logic is implemented in the gateway.

Chapter 5

REST Standards

All REST endpoints follow consistent naming conventions.

Examples:

```
GET    /api/v1/customers

GET    /api/v1/customers/{id}

POST   /api/v1/customers

PUT    /api/v1/customers/{id}

PATCH  /api/v1/customers/{id}

DELETE /api/v1/customers/{id}
```

Use nouns, not verbs.

Correct:

```
/customers
```

Incorrect:

```
/getCustomers
```

Chapter 6

Endpoint Design

Each endpoint has a single responsibility.

Examples:

```
POST /tickets

Creates a ticket
POST /tickets/{id}/close

Closes an existing ticket
```

Avoid endpoints performing multiple unrelated actions.

Chapter 7

API Versioning

API versions are part of the URL.

Example:

`/api/v1`

`/api/v2`

Breaking changes require a new major version.

Minor enhancements remain within the same version.

Chapter 8

Authentication

Authentication is based on secure tokens.

Supported mechanisms:

- JWT
- OAuth2
- API Keys (for integrations)

- Refresh Tokens

Passwords are never exposed through the API.

Chapter 9

Authorization

Authorization is role-based and policy-driven.

Examples:

- Administrator
- Manager
- Technician
- Customer
- AI Service
- Integration Service

Permissions are evaluated by the Policy Engine.

Chapter 10

Request Standards

Every request should include:

`Authorization`

`Content-Type`

`Accept`

`Accept-Language`

`X-Company-ID`

`X-Correlation-ID`

Optional:

`Idempotency-Key`

Chapter 11

Response Standards

All responses share a common structure.

Example:

```
{
  "success": true,
  "data": {},
  "metadata": {
    "requestId": "...",
    "timestamp": "...",
    "version": "v1"
  }
}
```

Errors follow the same pattern.

Chapter 12

Error Standards

Example:

```
{
  "success": false,
  "error": {
    "code": "CUSTOMER_NOT_FOUND",
    "message": "Customer not
found.",
    "details": [],
    "timestamp": "...",
    "requestId": "..."
  }
}
```

Error codes remain stable across versions.

Chapter 13

Pagination

Collection endpoints support pagination.

Example:

```
GET /customers?page=1&pageSize=25
```

Response metadata:

```
{
  "page": 1,
  "pageSize": 25,
  "totalItems": 1250,
  "totalPages": 50
}
```

Chapter 14

Filtering

Standard filtering syntax:

```
GET /tickets?status=open

GET /devices?brand=Motorola

GET /customers?language=es
```

Filters should be composable.

Chapter 15

Sorting

Standard syntax:

```
?sort=createdAt

?sort=-createdAt

?sort=priority
```

The minus sign indicates descending order.

Chapter 16

Search

Full-text search endpoints:

```
GET /search?q=laptop
```

```
GET /knowledge?q=battery
```

```
GET /customers?q=John
```

Search implementations are hidden behind the API contract.

Chapter 17

Idempotency

Operations creating resources may support idempotency.

Clients send:

`Idempotency-Key:`

Repeated requests with the same key return the original result.

This prevents duplicate tickets, invoices, or payments.

Chapter 18

Webhooks

External systems receive notifications through webhooks.

Examples:

`ticket.created`

`ticket.closed`

`payment.received`

`customer.created`

Webhook delivery includes retry logic and signature verification.

Chapter 19

WebSockets

Real-time communication uses WebSockets.

Typical use cases:

- Live chat
- Technician dashboard
- Ticket status updates
- Real-time notifications
- AI streaming responses

Connections must be authenticated.

Chapter 20

API Governance

Before publishing a new endpoint, architects should verify:

- Does a similar endpoint already exist?
- Is the resource name consistent?
- Does it follow REST conventions?
- Is it versioned correctly?
- Are security requirements satisfied?
- Is it documented?
- Are automated tests available?

No endpoint is released without governance review.

API Design Principles

The BiteFixes API Platform follows these principles:

- Contract First.
- Resource Oriented.
- Stateless.
- Secure by Default.
- Versioned.
- Observable.
- Backward Compatible.
- Multi-Tenant Aware.
- Language Independent.

Enterprise Vision

The API Platform is the public face of BiteFixes.

It allows mobile applications, web portals, AI services, partners and enterprise integrations to interact with the platform through a consistent and stable interface.

Internal implementation details may evolve, but the API contract remains predictable and well-governed.